ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о командном программном проекте на тему:

Robond: робот для автоматизированной торговли облигациями

Выполнили студенты:

Принял руководитель проекта:Куренков Владимир Вячеславович
Старший преподаватель
Факультет компьютерных наук НИУ ВШЭ

Москва 2026

Содержание

Аннотация . . . 3
Ключевые слова . . . 3
Введение . . . 4
Обзор литературы и существующих решений . . . 5
Архитектура системы . . . 7
- Модуль взаимодействия с API Московской биржи . . . 7
- Модуль получения исторических данных . . . 8
- База данных исторических данных . . . 8
- Модуль чтения исторических данных . . . 9
- Модуль тестирования стратегий на исторических данных . . . 9
- Модуль учёта портфеля . . . 10
- Модуль торговых стратегий . . . 10
  - Интерфейс стратегии . . . 10
  - Реализованные стратегии . . . 10
- Модуль взаимодействия с T-Invest API . . . 11
- Модуль работы в реальном времени . . . 11
- REST API-сервер . . . 12
- Веб-интерфейс . . . 12

Объёмные характеристики проекта . . . 13
Экспериментальный анализ стратегий . . . 13
- Условия эксперимента . . . 13
- Параметры стратегий . . . 13
- Результаты бэктестинга . . . 14
- Обсуждение результатов . . . 15

Заключение . . . 16
Список литературы . . . 17

Аннотация

В работе описывается разработка веб-приложения для алгоритмической торговли облигациями Московской биржи с возможностью тестирования торговых стратегий на исторических данных и последующей автоматической торговли через T-Invest API. Система ориентирована на частных инвесторов с различным уровнем подготовки и предусматривает выбор инвестиционной стратегии с учётом риск-профиля пользователя.

Проект реализован на языке программирования Rust (бэкенд, торговые алгоритмы, модуль бэктестинга) и TypeScript/React (веб-интерфейс). Система включает модули получения и хранения исторических данных, тестирования стратегий на исторических данных, учёта портфеля, формирования торговых решений на основе расчёта внутренней нормы доходности (XIRR) и взаимодействия с брокером по протоколу gRPC.

В результате работы реализованы все заявленные компоненты системы: модуль получения и хранения исторических данных по более чем 14 000 облигаций, модуль чтения данных из PostgreSQL, движок бэктестинга с симуляцией купонных выплат и погашений, три торговые стратегии с различными риск-профилями, gRPC-клиент T-Invest API, REST API-сервер с JWT-аутентификацией, модуль работы в реальном времени и веб-интерфейс. Проведён экспериментальный анализ стратегий на исторических данных за период с января 2024 по март 2026 года; стратегия YieldMaximizer показала доходность 81,9% за 27 месяцев. Общий объём кода составляет около 6800 строк (5679 строк Rust, 1128 строк TypeScript).

Ключевые слова

Алгоритмическая торговля, облигации, Московская биржа, торговый робот, бэктестинг, Rust, T-Invest API, gRPC, XIRR, риск-профиль, PostgreSQL.

Введение

Алгоритмическая торговля является важным направлением развития финансовых технологий. Современные рынки характеризуются высокой скоростью обработки информации и большим объёмом данных, что делает автоматизированные методы анализа и принятия решений особенно актуальными. При этом большинство существующих решений либо ориентировано на профессиональных участников рынка, либо требует серьёзной технической подготовки.

Рынок облигаций представляет интерес как инструмент более консервативного инвестирования по сравнению с рынком акций. Однако торговля облигациями требует учёта специфических характеристик: купонных выплат, сроков до погашения, номинала, доходности к погашению, кредитного рейтинга и риска дефолта. Эти особенности усложняют разработку торговых алгоритмов и требуют специализированной инфраструктуры.

Целью данного проекта является разработка веб-приложения для алгоритмической торговли облигациями Московской биржи, которое:

- предоставляет пользователю возможность выбора инвестиционной стратегии с учётом риск-профиля (консервативная, умеренная, агрессивная);
- автоматически формирует портфель облигаций на основе выбранной стратегии;
- позволяет тестировать стратегию на исторических данных с учётом купонных выплат, амортизации и погашения номинала;
- обеспечивает автоматическую торговлю через T-Invest API.

Проект реализован командой из трёх человек. Структура распределения задач следующая:

- **Брыкульская Мария** — модуль чтения исторических данных, движок бэктестинга, модуль учёта портфеля, веб-интерфейс;
- **Широковских Александр** — модуль торговых стратегий, gRPC-клиент T-Invest API, модуль работы в реальном времени;
- **Лобанов Максим** — модуль взаимодействия с API Московской биржи, модуль получения исторических данных, база данных.

В данном отчёте описывается итоговый результат проекта: архитектура системы, описание всех реализованных модулей, обзор релевантной литературы, результаты экспериментального анализа стратегий и заключение.

Обзор литературы и существующих решений

В академической литературе широко рассматриваются задачи портфельной оптимизации, управления риском и алгоритмической торговли. Классическая теория портфеля Марковица [1] предполагает оптимизацию соотношения доходности и риска на основе ковариационной матрицы активов. Однако данный подход ориентирован преимущественно на акции и не учитывает специфику облигаций: купонные выплаты, амортизацию номинала и дискретные сроки до погашения. Кроме того, практическое применение модели Марковица затруднено из-за чувствительности оценок ковариационной матрицы к малым изменениям входных данных.

Для оценки доходности облигаций широко используется метрика XIRR (Extended Internal Rate of Return) — расширенная внутренняя норма доходности, учитывающая произвольные даты денежных потоков [2]. В отличие от доходности к погашению (YTM), XIRR позволяет корректно оценивать доходность с учётом фактических дат покупки и всех последующих выплат, что особенно важно для облигаций с нерегулярными купонами и амортизацией.

Существуют готовые платформы для алгоритмической торговли. Backtrader [3] — библиотека для тестирования стратегий на языке Python, предоставляющая гибкий фреймворк для симуляции. Однако библиотека ориентирована на рынок акций и не имеет встроенных инструментов для моделирования купонных выплат. QuantConnect [4] предоставляет облачную платформу для разработки и тестирования алгоритмических стратегий с поддержкой множества рынков, но не имеет специализированных модулей для российского рынка облигаций. MetaTrader [5] — платформа, широко используемая на практике, ориентирована на рынок Forex и CFD и не предоставляет интеграции с Московской биржей через API.

T-Invest API [6] предоставляет программный интерфейс для получения рыночных данных и совершения сделок через брокера Т-Инвестиции. API реализовано на базе протокола gRPC и поддерживает работу с акциями, облигациями, валютами, ETF и другими инструментами. Однако API не включает встроенные средства для тестирования стратегий на исторических данных и не предоставляет инструментов для корректного моделирования специфики облигационного рынка (купоны, амортизация, оферты, дефолты).

Открытый API Московской биржи [7] предоставляет доступ к историческим котировкам, информации об инструментах и статическим характеристикам облигаций. Данные доступны в формате JSON через HTTP-запросы и покрывают широкий спектр инструментов, включая все торгуемые облигации.

Таким образом, разработанная система отличается от существующих решений следующими аспектами:

- **специализация под рынок облигаций Московской биржи** — корректное моделирование купонных выплат, амортизации, погашения номинала, оферт и дефолтов;
- **единая архитектура для тестирования и реальной торговли** — одни и те же стратегии используются в движке бэктестинга и в модуле работы в реальном времени через общий трейт `Strategy`;

- **веб-интерфейс для частных инвесторов** — возможность выбора стратегии, просмотра результатов тестирования и подключения автоматической торговли без необходимости программирования;
- **реализация на языке Rust** — обеспечение высокой производительности и безопасности при обработке больших объёмов данных и многопоточном выполнении.

Архитектура системы

Система имеет модульную архитектуру, состоящую из десяти компонентов. Общая схема взаимодействия модулей представлена на Рисунке 1.

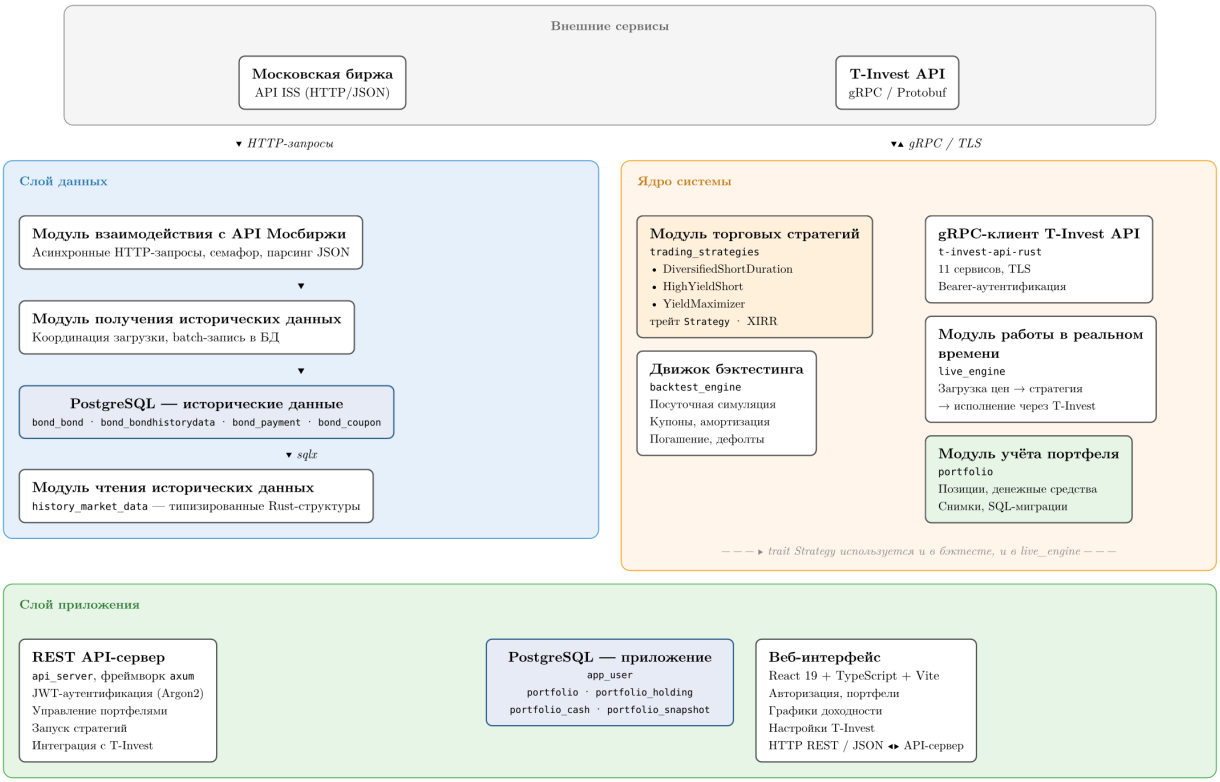

Рис. 1. Общая архитектура веб-приложения алгоритмической торговли облигациями.

Ниже описаны все модули системы, их функциональность и ключевые технические решения.

Модуль взаимодействия с API Московской биржи

Данный модуль предназначен для получения исторических котировок в формате свечей и статической дополнительной информации по облигациям путём выполнения HTTP-запросов к открытому API Московской биржи [7].

Функциональность модуля включает:

- получение дневных свечей (OHLCV) за указанный период;
- получение статической информации об облигациях: ISIN, номинал, дата погашения, дата размещения, объём выпуска, доходность и др.;
- получение информации о купонных выплатах, амортизации и погашении номинала;
- фильтрация данных по дефолтам и офертам.

В процессе разработки потребовалась оптимизация модуля, поскольку необходимо было собрать исторические данные по более чем 14 тысячам облигаций. Наивная реализация, выполняющая последовательные HTTP-запросы и отдельные операции записи, приводила к неприемлемо высоким временным затратам. Оптимизация была выполнена двумя способами:

- **Асинхронные запросы.** Для контроля нагрузки на внешний сервис применялся паттерн семафор, ограничивающий количество одновременных запросов.
- **Пакетная (batch) запись в базу данных.** Полученные данные накапливаются в буфер и записываются в базу данных одной групповой операцией.

Модуль написан Максимом Лобановым.

Модуль получения исторических данных

Модуль получения исторических данных выполняет роль координатора: используя модуль взаимодействия с API Московской биржи (раздел 3.1), он собирает котировки и статическую информацию об облигациях за заданный исторический период и записывает их в базу данных PostgreSQL.

Модуль написан Максимом Лобановым.

База данных исторических данных

Реализована на PostgreSQL и хранит исторические данные облигаций. Основными таблицами являются:

Таблица `bond_bond` — содержит статическую информацию об облигациях:

- `id` — уникальный идентификатор облигации;
- `isin` — международный идентификатор ценной бумаги;
- `title` — наименование облигации;
- `maturity_date` — дата погашения;
- `facevalue` — номинальная стоимость;
- `issue_volume` — объём выпуска;
- `is_traded` — признак торгуемости;
- `is_subordinated` — признак субординированности;
- `board` — режим торгов.

Таблица `bond_bondhistorydata` — содержит исторические торговые данные:

- `date` — дата торгов;
- `open`, `close`, `low`, `high` — цены за торговый день (в процентах от номинала);
- `volume` — объём торгов;
- `num_trades` — количество сделок;
- `accint` — накопленный купонный доход (НКД);
- `facevalue` — номинал на дату торгов;
- `bond_id` — внешний ключ на таблицу `bond_bond`.

Таблица `bond_payment` — содержит информацию о выплатах:

- `date` — дата выплаты;
- `size` — размер выплаты;
- `type_id` — тип выплаты (1 — амортизация, 2 — купон, 14 — погашение номинала, другие — оферты).

Таблица `bond_coupon` — содержит информацию о купонах:

- `size` — размер купона;
- `period` — период выплаты;
- `aci` — текущий НКД.

В таблице исторических данных реализованы индексы по полю `bond_id` и составной индекс по полям `(date, bond_id)`, что ускоряет как получение истории конкретной облигации, так и основной тип запроса при бэктестинге — получение всех облигаций за конкретную дату.

Реализовал Максим Лобанов.

Модуль чтения исторических данных

Модуль чтения исторических данных (`history_market_data`) предназначен для извлечения информации из базы данных PostgreSQL и преобразования её во внутренние структуры Rust, используемые другими компонентами системы. Реализован с применением библиотеки `sqlx` и асинхронного runtime `tokio`.

Функциональность модуля включает:

- получение всех свечей за заданную дату;
- получение исторических данных для конкретной облигации за указанный период;
- получение всех свечей в заданном диапазоне дат (оптимизированный запрос без загрузки JSON-полей);
- получение полной информации об облигации;
- получение информации о купонных выплатах (с группировкой по `bond_id` в `HashMap`);
- получение выплат (купоны, амортизация, погашение) за заданный период;
- получение дат оферт и дефолтов для фильтрации при бэктестинге.

Исторические данные представлены структурой `BondHistoryData`, содержащей информацию о параметрах торгов за день. Статическая информация — структурой `BondInfo`, включающей ISIN, даты, доходность и другие характеристики. Результаты возвращаются в виде типизированных структур, что позволяет использовать их в модуле бэктестинга и модуле учёта портфеля без дополнительной обработки.

Модуль реализован Марией Брыкульской.

Модуль тестирования стратегий на исторических данных

Модуль тестирования стратегий (`backtest_engine`) реализует симуляцию работы торговой стратегии на исторических данных Московской биржи. Движок выполняет посуточную итерацию по заданному временному диапазону и для каждого торгового дня:

1. загружает актуальные рыночные цены из базы данных;
2. обрабатывает купонные выплаты, амортизацию и погашение номинала;
3. фильтрует облигации, достигшие даты оферты или дефолта;
4. вызывает стратегию для формирования торговых поручений;
5. симулирует исполнение сделок;
6. фиксирует снимок состояния портфеля.

Ключевые структуры движка:

- **`BacktestEngine`** — основной оркестратор, хранящий ссылку на клиент исторических данных, начальный капитал, даты начала и конца тестирования;
- **`MarketSimulator`** — ядро симуляции, поддерживающее текущую дату, состояние портфеля, кеш цен, историю сделок и выплат;
- **`BacktestResult`** — итоговый результат: начальный капитал, финальная стоимость, доходность, список сделок, выплат и ежедневных снимков портфеля.

Симуляция сделок осуществляется по средней цене свечи $\frac{\text{low} + \text{high}}{2}$ с учётом НКД. При покупке проверяется наличие свободных средств и дневной объём торгов. При продаже проверяется наличие достаточного количества облигаций в портфеле.

Модуль покрыт модульными тестами: проверка корректности исполнения покупки (списание средств), обработка ошибки при продаже отсутствующих бумаг, начисление купонных выплат, точность расчёта стоимости портфеля.

Модуль реализован Марией Брыкульской.

Модуль учёта портфеля

Модуль учёта портфеля (`portfolio`) обеспечивает персистентное хранение состояния портфеля в базе данных PostgreSQL. Схема данных определяется четырьмя SQL-миграциями.

Функциональность модуля:

- хранение портфелей пользователей с привязкой к аккаунту (`user_id`);
- хранение позиций — количество облигаций каждого типа по ISIN (с семантикой upsert);
- управление свободными денежными средствами с точностью до 6 знаков после запятой (`NUMERIC(20, 6)`);
- фиксация снимков портфеля (дата, рыночная стоимость, денежные средства, стоимость облигаций) для построения графика доходности;
- привязка торговой стратегии к портфелю;
- хранение учётных данных T-Invest API (токен, идентификатор счёта, контур).

Модуль реализован Марией Брыкульской.

Модуль торговых стратегий

Модуль торговых стратегий (`trading_strategies`) описывает правила принятия решений о покупке и продаже облигаций. Модуль определяет общий интерфейс через трейт `Strategy` с единственным методом `decide_trades`, и предоставляет несколько конкретных реализаций.

Интерфейс стратегии

Трейт `Strategy` принимает на вход:

- текущую дату;
- состояние портфеля (свободные средства и количество облигаций по ISIN);
- справочную информацию по облигациям (даты и суммы выплат, характеристики);
- текущие рыночные цены и объёмы торгов.

На выходе стратегия возвращает вектор рыночных поручений `MarketOrder`, каждое из которых содержит ISIN, тип операции (покупка/продажа) и количество.

Для оценки привлекательности облигаций все стратегии используют метрику XIRR (расширенная внутренняя норма доходности), рассчитываемую на основе текущей цены покупки и всех будущих денежных потоков (купоны, амортизация, погашение номинала). Расчёт XIRR выполняется с помощью библиотеки `financial`.

Реализованные стратегии

DiversifiedShortDurationStrategy — консервативная стратегия, покупающая облигации с дюрацией 3–18 месяцев по сниженной цене. Портфель диверсифицируется: минимум

5 облигаций, не более 15% на одну позицию. Реализован стоп-лосс при падении цены ниже 70% от номинала. Облигации ранжируются по XIRR, покупаются наиболее доходные.

HighYieldShortStrategy — агрессивная стратегия, ориентированная на облигации с XIRR $\geq$ 22%. Короткая дюрация (14–400 дней), максимальная доля одной позиции 8%. Более жёсткие фильтры по цене (60–120% от номинала).

YieldMaximizerStrategy — стратегия максимизации XIRR с жёстким дедлайном по дате погашения. Не использует стоп-лосс (удерживает облигации до погашения). Динамический порог доходности: при доле свободных средств свыше 30% порог снижается для размещения простаивающего капитала. Максимальная доля одной позиции 5%.

Модуль реализован Александром Широковских.

Модуль взаимодействия с T-Invest API

Модуль `t-invest-api-rust` предназначен для взаимодействия с инвестиционным API T-Инвестиций по протоколу gRPC и предоставляет типизированный клиент на языке Rust для вызова методов API. Модуль извлекает контракты из официальных protobuf-описаний, генерирует типы и клиентов сервисов и даёт единую точку доступа к сервисам API с автоматической подстановкой токена авторизации.

Функциональность модуля включает:

- **работа с инструментами** — получение информации об облигациях, акциях, ETF, поиск по идентификатору или тикеру;
- **рыночные данные** — получение последних цен, стакана заявок, исторических свечей;
- **операции и портфель** — получение списка операций по счёту, позиций и портфеля;
- **заявки** — выставление, отмена и изменение рыночных и лимитных заявок;
- **потоковые сервисы** — подписка на стримы операций и заявок;
- **песочница** — работа с тестовым контуром без реальных средств.

Клиент API представлен структурой `Client`, объединяющей клиентов всех 11 сервисов. Выбор контура задаётся перечислением `EndPoint` (`Prod` — боевой сервер, `Sandbox` — песочница). Все запросы автоматически дополняются заголовком авторизации `Bearer` через `AuthorizationInterceptor`.

Модуль реализован с использованием gRPC-фреймворка `tonic` и библиотеки `prost`. Сборка контрактов выполняется на этапе компиляции через `build.rs`. Одно установленное TLS-соединение (`Channel`) переиспользуется всеми клиентами сервисов. Для конвертации типов предоставляются утилиты `quotation_to_decimal` и `decimal_to_quotation`.

Модуль реализован Александром Широковских.

Модуль работы в реальном времени

Модуль `live_engine` реализует цикл автоматической торговли:

1. загрузка информации обо всех рублёвых облигациях через T-Invest API;
2. получение текущего состояния портфеля (позиции и свободные средства);
3. расчёт рыночных цен облигаций с учётом номинала и НКД;
4. вызов стратегии для формирования списка поручений;
5. исполнение сделок через T-Invest API.

Ключевая особенность модуля — использование того же трейта `Strategy`, что и в движке бэктестинга, что обеспечивает полную совместимость стратегий между режимами тестирования и реальной торговли.

Модуль реализован Александром Широковских.

REST API-сервер

REST API-сервер (`api_server`) построен на фреймворке `axum` и обеспечивает взаимодействие веб-интерфейса с бэкендом. Сервер предоставляет следующие группы эндпоинтов:

Аутентификация:

- регистрация (`POST /api/auth/signup`), вход (`POST /api/auth/login`), обновление токена (`POST /api/auth/refresh`);
- JWT-аутентификация: access-токен (15 мин), refresh-токен (7 дней);
- хеширование паролей с использованием Argon2.

Управление портфелями:

- CRUD-операции над портфелями, позициями и денежными средствами;
- снимки портфеля для построения графика доходности;
- расчёт общей доходности.

Торговые стратегии:

- назначение стратегии портфелю;
- запуск стратегии на реальном счёте T-Invest;
- получение списка доступных стратегий.

Интеграция с T-Invest:

- подключение и отключение токена T-Invest;
- импорт портфеля из T-Invest;
- проверка статуса подключения.

Все запросы к защищённым эндпоинтам проходят через middleware `AuthUser`, извлекающий `user_id` из JWT-токена. Портфели изолированы по пользователям.

Модуль реализован совместно всеми участниками команды.

Веб-интерфейс

Веб-интерфейс реализован как одностраничное приложение (SPA) на React 19 с TypeScript. Для маршрутизации используется React Router, для управления серверным состоянием — TanStack React Query, для HTTP-запросов — Axios. Сборка выполняется с помощью Vite.

Интерфейс включает следующие страницы:

- **Авторизация** — вход и регистрация;
- **Список портфелей** — просмотр портфелей пользователя, создание нового;
- **Детали портфеля** — позиции, денежные средства, график стоимости, управление стратегией;
- **Настройки** — подключение T-Invest API, выбор счёта.

Доступ к защищённым страницам ограничен компонентом `ProtectedRoute`, проверяющим наличие JWT-токена.

Модуль реализован Марией Брыкульской.

Объёмные характеристики проекта

Компонент	Язык	Строк кода
`backtest_engine`	Rust	1400
`history_market_data`	Rust	550
`portfolio`	Rust	370
`trading_strategies`	Rust	900
`t-invest-api-rust`	Rust	360
`api_server`	Rust	800
`live_engine`	Rust	210
SQL-миграции	SQL	60
`web` (фронтенд)	TypeScript	1130
Итого		**5780**

Таблица 1. Объёмные характеристики кодовой базы проекта.

Кроме того, в базе данных исторических данных хранятся котировки более чем 14 000 облигаций Московской биржи.

Экспериментальный анализ стратегий

Для оценки эффективности реализованных торговых стратегий был проведён бэктестинг на исторических данных облигаций Московской биржи.

Условия эксперимента

Тестирование проводилось при следующих условиях:

- **Период:** 1 января 2024 — 31 марта 2026 (27 месяцев);
- **Начальный капитал:** 1 000 000 рублей;
- **База данных:** более 14 000 облигаций Московской биржи;
- **Модель исполнения:** покупка и продажа по средней цене свечи $\frac{\text{low} + \text{high}}{2}$ с учётом НКД;
- **Ограничения:** проверка дневного объёма торгов, объёма выпуска, наличия свободных средств;
- **Учёт выплат:** купоны, амортизация и погашение номинала начисляются в дату выплаты;
- **Фильтрация:** облигации с дефолтом и офертой исключаются из торговли с соответствующей даты.

Параметры стратегий

В Таблице 2 представлены ключевые параметры трёх тестируемых стратегий.

Параметр	Diversified	HighYield	YieldMax
Макс. доля позиции	15%	8%	5%
Мин. дней до погашения	90	14	14
Макс. дней до погашения	540	400	—
Мин. цена (% от номинала)	50%	60%	75%
Макс. цена (% от номинала)	—	120%	115%
Стоп-лосс	70%	70%	нет
Мин. XIRR	5%	22%	1%
Мин. объём для покупки	—	—	50 лотов

Таблица 2. Ключевые параметры реализованных торговых стратегий.

Стратегия DiversifiedShortDuration ориентирована на консервативный профиль: широкая диверсификация, умеренный порог доходности и стоп-лосс для защиты от убытков. HighYieldShort нацелена на агрессивный профиль с высоким порогом доходности (XIRR $\geq$ 22%). YieldMaximizer занимает промежуточное положение: минимальный порог XIRR, но жёсткий контроль размера позиции и динамический порог при избытке свободных средств.

Результаты бэктестинга

В качестве основного эксперимента была протестирована стратегия YieldMaximizer на полном 27-месячном периоде. Результаты представлены в Таблице 3.

Метрика	Значение
Начальный капитал	1 000 000 руб.
Финальная стоимость портфеля	1 818 972 руб.
Прибыль	818 972 руб.
Доходность за период	81,9%
Годовая доходность (приблизительно)	30%
Количество сделок	875
Количество уникальных облигаций	80+

Таблица 3. Результаты бэктестинга стратегии YieldMaximizer (январь 2024 — март 2026).

Стратегия продемонстрировала доходность 81,9% за 27 месяцев, что соответствует приблизительно 30% годовых. За период тестирования было совершено около 875 сделок с более чем 80 уникальными облигациями. Портфель на протяжении всего периода поддерживал высокую диверсификацию (более 20 позиций одновременно).

Ключевые особенности поведения стратегии YieldMaximizer:

- **жёсткий дедлайн** по дате погашения (31 марта 2026) исключил из портфеля долгосрочные облигации, подверженные процентному риску;
- **динамический порог доходности** обеспечил размещение простаивающего капитала при доле свободных средств свыше 30%;
- **отсутствие стоп-лосса** позволило удерживать облигации до погашения, избегая фиксации убытков при временных снижениях цены;

- **малая доля одной позиции** (не более 5%) обеспечила устойчивость к дефолтам отдельных эмитентов.

В качестве базового сравнения также был проведён бэктест стратегии DoNothing (без совершения сделок — аналог банковского депозита с нулевой ставкой), подтвердивший корректность работы движка: портфель не изменился за период тестирования.

Обсуждение результатов

Полученная доходность (30% годовых) существенно превышает среднюю ставку по банковским депозитам в рассматриваемый период. Следует отметить, что бэктестинг проводился без учёта комиссий брокера и налогов, что может снизить реальную доходность на 2–5 процентных пунктов.

Высокая доходность стратегии YieldMaximizer объясняется в том числе специфическим рыночным окружением 2024–2026 годов: период повышенных ключевых ставок создал благоприятные условия для покупки краткосрочных облигаций со значительным дисконтом к номиналу. Для более полной оценки устойчивости стратегии необходимо тестирование на более длительных периодах, включающих различные фазы экономического цикла.

Заключение

В ходе выполнения проекта разработано веб-приложение Robond для алгоритмической торговли облигациями Московской биржи. Реализованы все заявленные компоненты системы:

1. **Инфраструктура данных:** модуль взаимодействия с API Московской биржи, модуль получения и хранения исторических данных по более чем 14 000 облигаций в базе PostgreSQL, модуль чтения данных с типизированными структурами Rust.
2. **Движок бэктестинга:** посуточная симуляция с корректным моделированием купонных выплат, амортизации, погашения номинала, оферт и дефолтов. Модуль покрыт модульными тестами.
3. **Торговые стратегии:** реализованы три стратегии с различными риск-профилями (консервативный, агрессивный, оптимизационный), использующие метрику XIRR для оценки привлекательности облигаций.
4. **Интеграция с брокером:** gRPC-клиент T-Invest API с поддержкой всех 11 сервисов, модуль работы в реальном времени, использующий тот же трейт `Strategy`, что и движок бэктестинга.
5. **Веб-приложение:** REST API-сервер на `axum` с JWT-аутентификацией, управлением портфелями и интеграцией с T-Invest; фронтенд на React/TypeScript.

Проведён экспериментальный анализ стратегий на исторических данных за период январь 2024 — март 2026. Стратегия YieldMaximizer показала доходность 81,9% за 27 месяцев (30% годовых) при начальном капитале 1 млн рублей, совершив около 875 сделок с более чем 80 облигациями.

Основным результатом проекта является работающая система, объединяющая тестирование и реальную торговлю в единой архитектуре через общий интерфейс стратегий. Общий объём кода составляет около 6800 строк.

Направления дальнейшего развития:

- добавление комиссий брокера и проскальзывания в модель бэктестинга;
- реализация стратегии на основе кредитного рейтинга;
- расширенный экспериментальный анализ: сравнение стратегий по коэффициенту Шарпа и максимальной просадке;
- тестирование на более длительных исторических периодах.

Ссылка на репозиторий: https://github.com/robond-fintech/robond.

Список литературы

[1] H. Markowitz, «Portfolio Selection», *The Journal of Finance*, т. 7, вып. 1, сс. 77–91, 1952.

[2] A. Damodaran, *Investment Valuation: Tools and Techniques for Determining the Value of Any Asset*, 3-е изд. John Wiley & Sons, 2012.

[3] Backtrader Community, «Backtrader: Python Backtesting Library». [Онлайн]. Доступно на: https://www.backtrader.com/

[4] QuantConnect, «QuantConnect Algorithmic Trading Platform». [Онлайн]. Доступно на: https://www.quantconnect.com/

[5] MetaQuotes Ltd., «MetaTrader 5 Trading Platform». [Онлайн]. Доступно на: https://www.metatrader5.com/

[6] T-Bank, «T-Invest API Documentation». [Онлайн]. Доступно на: https://tinkoff.github.io/investAPI/

[7] ПАО Московская биржа, «Информационно-статистический сервер Московской биржи (ISS)». [Онлайн]. Доступно на: https://iss.moex.com/